Development Plan for Issue #300
Tensor-Native Trace Discretization and Skeleton Support

AI-assisted development plan

May 29, 2026

1 Objective

We aim to introduce the next missing topology/discretization/runtime layer in the tensor-native stack: explicit support for trace variables together with the tensor-native skeletons on which they live. The immediate goal is *not* full trace-term operator realization. Rather, the purpose of the present work is to define a clean geometric and runtime substrate that fits the current tensor-native architecture and leaves a direct path to later trace-aware local apply and operator-factory work.

For the current orthogonal 1D component setting, the intended interpretation is dimension-specific. Given a chosen tensor component dimension d, the associated traces should live on the part of the tensor mesh skeleton orthogonal to that dimension. Concretely, we want this support to be obtained by:

1. taking the skeleton in component dimension d, namely the active component vertices in that dimension, and
2. tensoring that vertex-fixed support with the remaining component topologies through the retained active-cell tuples of the corresponding slices.

Thus, for example, a spatial trace in a 1D space $\times$ 1D time $\times$ 1V tensor mesh should live on spatial endpoints tensored with time and velocity topology, while a time trace should live on time-slab endpoints tensored with the spatial and velocity component topologies.

2 Scope and Constraints

The intended scope of issue #300 is deliberately structural. In the present work, we want to define the topology, discretization, local-layout, and runtime interpretation for traces, but we do *not* yet want to solve the full side-term realization problem.

The main constraints appear to be the following:

1. We should build directly on the current `TensorMeshTopology`, `TensorMesh`, `TensorVar`, `TensorVarDiscretiza` `TensorDofLayout`, and `TensorGatherScatter` abstractions rather than introducing a parallel trace-only stack.
2. We should keep the geometry tensor-native and should not fall back to the legacy `MeshTopology`-subclass or full-dimensional mesh-entity model as the defining abstraction.
3. We should preserve the current interpretation that the tensor-native local coefficient space is explicit and local, with deterministic flattenings/orderings rather than hidden legacy GDA-style machinery.

4. We should leave a clean path toward later trace-aware operator realization, including side quadrature, trace-local basis selection, and eventual matrix-free or PAMatrix-aware backends.

5. We should remain conservative with public API growth: the new trace and skeleton support should be explicit, but we should avoid premature generalization beyond what the present 1D-orthogonal-component setting requires.

3 Current State and the Main Gap

Issue #299 established the first genuinely finite-element-aware tensor-native volume apply path. At present, however, the tensor-native stack is still essentially volume-variable-centric. The current code gives us:

- componentwise 1D topology with active cells, active vertices, and vertex–cell incidence,
- full-product tensor cell tuples in `TensorMeshTopology`,
- tensor-native trial/test variable registration in `TensorMesh`,
- basic field-versus-trace categorization in `TensorVar`,
- per-variable discretization metadata in `TensorVarDiscretization`,
- local tensor coefficient layouts in `TensorDofLayout`, and
- flat-buffer gather/scatter in `TensorGatherScatter`.

What is still missing is a tensor-native notion of trace support that answers the following questions explicitly:

- If a variable is a trace, *which* tensor component dimension is it associated with?
- On what tensor-native skeleton does it live?
- How are the relevant trace-support entities indexed and iterated deterministically?
- How should the local coefficient layout for a trace variable be interpreted relative to that skeleton support?

This is the central gap that issue #300 is meant to fill.

4 Working Assumptions from the Current Code

Based on the current source context, we proceed under the following assumptions.

4.1 Component skeleton information is already nearly available

`ComponentMeshTopology1D` already stores active vertices, cell-to-vertex incidence, and vertex-to-cell incidence. In particular, it already exposes:

- `vertexCount()`,
- `cellVertexIndex(...)`,

- `activeVertexCoordinates()`, and
- host-accessible vertex-to-cell incidence offsets and ordinals.

This means that for the current 1D setting, the relevant component skeleton is naturally identified with the active component vertices. We therefore do not need a fundamentally new component-level topology object for issue #300, though we may benefit from a small convenience query or two.

4.2 Tensor topology already supports vertex-fixed slicing

`TensorMeshTopology::slice(...)` already takes fixed component vertices and returns the retained lower-dimensional tensor topology induced by cells incident on those vertices. This is already very close to the desired geometric operation for traces: fixing one component to a skeleton vertex and tensoring that support with the remaining component topologies through the retained active-cell tuples.

Accordingly, one promising design direction is to make the trace-support geometry explicit in `TensorMeshTopology` by introducing a dimension-specific tensor-skeleton view built from the relevant component vertices and the existing slice logic.

4.3 Current trace semantics are not yet dimension-specific

`TensorVar` currently distinguishes only `TENSOR_FIELD` versus `TENSOR_TRACE`. It validates that field variables wrap legacy `FIELD` vars and trace variables wrap legacy `TRACE` or `FLUX` vars. However, it does not yet record which tensor component dimension a trace belongs to.

This is a key missing piece. The current code already suggests that `TensorVar` is the right home for semantic trace identity, so the likely next step is to extend `TensorVar` with an explicit trace-component ordinal for trace variables.

4.4 Current local layouts are variable-local tensor products

`TensorDofLayout` currently builds one `VariableLayout` per registered variable, using one component cardinality per tensor component and one stride per tensor component. This works well for volume variables, where the local coefficient block is modeled as a tensor product over component basis cardinalities.

For traces, however, one component dimension should contribute *skeleton support entities* rather than a volume cell basis cardinality. The remaining component dimensions should continue to contribute their volume-style component discretization information. This suggests that issue #300 should extend the local-layout metadata so that each component can be classified by support kind rather than assuming pure volume basis cardinality everywhere.

5 Proposed Geometric Model

The central design decision is to represent a trace support associated with component dimension d as a tensor-native skeleton object whose entities are indexed by retained active-cell tuples in the slice induced by fixing a component-d active vertex. Equivalently, we may index such an entity by

$$(v_d, K_0, \ldots, K_{d-1}, K_{d+1}, \ldots, K_{n-1}),$$

with the understanding that the retained tuple appears once for each slice-supported incident configuration at the chosen vertex v_d.

In other words, the trace-support domain is obtained by taking the active-vertex set V_d in component d and, for each such vertex, pairing it with the retained active-cell tuples of the corresponding slice. In the common boundary case this agrees with the informal shorthand

$$\mathcal{S}_d = V_d \times \prod_{r \neq d} \mathcal{T}_r,$$

but for interior vertices the slice interpretation is more precise because multiple incident retained tuples may occur.

This is the intended implementation-level reading of the "component-d skeleton tensored with the remaining component mesh topologies" interpretation described in the issue text.

5.1 Tensor-skeleton entity interpretation

Each trace-support entity should be thought of as a codimension-one tensor-product interface orthogonal to dimension d. In the current 1D-component setting, this means that fixing a vertex in component d selects an endpoint in that dimension, while the other dimensions remain volumetric.

This interpretation has two useful consequences:

1. it gives a deterministic indexing model for trace support entities, and
2. it aligns naturally with later side-term realization, since each such entity corresponds to a lower-dimensional product domain on which one would eventually perform trace or flux integration.

5.2 Boundary versus interior trace-support entities

At the topology level, some trace-support entities correspond to boundary support and others to interior support. In the present work, we do not need full neighbor pairing or orientation logic for side terms, but it is worth exposing enough metadata to distinguish whether a skeleton vertex in component dimension d is incident to one active cell or more than one. The current component-topology incidence data is already sufficient for this.

6 Proposed API and Data-Model Direction

6.1 1. Extend `TensorVar` with trace-component semantics

We propose that `TensorVar` remain the semantic home for dimension-specific trace identity. In particular, a trace variable should carry an explicit *trace component ordinal*. Field variables should remain dimension-neutral in this sense.

A likely public interface shape is:

- existing constructor for field variables remains available,
- a trace-aware constructor or factory path records the trace component ordinal,
- `hasTraceComponentOrdinal()` or equivalent query,
- `traceComponentOrdinal()` with validation for trace variables.

We should also strengthen validation so that:

- field variables must not carry a trace component ordinal, and

- trace variables must carry exactly one valid trace component ordinal in range.

This keeps the dimension-specific trace meaning explicit and attached to the variable descriptor itself, which appears to be the cleanest fit with the current design.

6.2 2. Extend `TensorMeshTopology` with explicit tensor-skeleton support

We propose adding an explicit helper type in `TensorMeshTopology`, likely something like a `TensorSkeleton` or `TraceSupport` view. The purpose of this type would be to expose:

- the chosen orthogonal component dimension d,
- the retained component topologies in the remaining dimensions,
- the component-d active vertices participating in the skeleton,
- deterministic trace-support entity tuples, and
- convenience queries for lower-dimensional sliced topologies or incident cells.

For the first pass, this helper can remain relatively lightweight and topology-local. The crucial point is that the tensor-native skeleton should become an explicit object rather than remaining an implicit pattern of repeated `slice(...)` calls.

A representative indexing object might take the form:

$$\texttt{TensorSkeletonEntity} = (d, v_d, K_{r_1}, \ldots, K_{r_m}),$$

where the retained cells follow the current lower-indexed-components-vary-fastest convention after omitting component d, and where multiple retained tuples may arise at an interior vertex through slice incidence.

6.3 3. Extend `TensorDofLayout::VariableLayout` with support-kind metadata

For volume variables, each component contributes a basis cardinality. For trace variables, one distinguished component contributes a *skeleton-support cardinality*, which in the first implementation is represented by the active component-vertex extent, while the remaining components contribute their usual discretization-side basis cardinalities.

Accordingly, `VariableLayout` should likely grow per-component support metadata, for example along the lines of:

- whether a component contributes volume basis ordinals or trace-support ordinals,
- the distinguished trace component ordinal, if any,
- per-component extents for the local flattening, and
- possibly a small helper query layer such as `isTraceSupportComponent(d)`.

The key point is that trace-local layout should still be a deterministic flattening of one variable-local tensor coefficient space, but its factors are no longer all "basis cardinality on a component cell."

6.4 4. Keep `TensorVarDiscretization` structurally light unless needed

At present, `TensorVarDiscretization` stores basis-family override and per-component polynomial-order deltas. Based on the current code, it does not appear necessary to move trace-component identity into this object. The trace component belongs more naturally to the semantic variable itself.

However, we should ensure that discretization and layout construction can interpret trace variables correctly. If later work needs trace-basis-family overrides or trace-side per-component deltas with special meaning, that can be added then. For issue #300, the current discretization object likely needs at most minor validation or comment updates.

6.5 5. Preserve flat-buffer gather/scatter compatibility

`TensorGatherScatter` is currently a thin flat-buffer operator driven by `TensorDofLayout`. If `TensorDofLayout` correctly reflects the trace-local coefficient-space size and ordinal maps, then `TensorGatherScatter` may require little or no behavioral change in this issue. It should continue to work as a flat local/global mover, provided that localDofCount and local ordinal interpretation are updated coherently for trace variables.

This is desirable, because it allows issue #300 to remain focused on support geometry and local layout rather than expanding unnecessarily into new runtime operator mechanics.

7 Algorithmic Approach

7.1 Tensor-skeleton enumeration

For a chosen trace dimension d, we propose to enumerate tensor-skeleton entities by iterating over:

1. active vertices in component d, and
2. the retained active-cell tuples in the slice induced by fixing that vertex.

using the same lower-indexed-components-vary-fastest convention already used elsewhere in the tensor-native stack.

Equivalently, we may describe the same construction as taking each active vertex v_d and forming the slice topology induced by fixing component d at that vertex, then enumerating the active retained tuples of that slice. This is the clearest statement of the implemented semantics, and it explains naturally why interior component vertices may contribute more than one trace-support entity.

7.2 Trace-local layout construction

For a trace variable associated with component d, the local layout should be constructed as follows.

1. For component d, use an extent corresponding to the relevant skeleton-support cardinality. In the first implementation, this is the active component-vertex extent in that dimension.
2. For each component $r \neq d$, compute the same effective basis cardinality that volume variables currently use.
3. Form the total local dof count as the product of these per-component extents.

4. Build variable-ordering strides with lower-indexed retained tensor factors varying fastest, consistent with current tensor-local conventions.

In effect, the trace-local coefficient space becomes a hybrid tensor product:

$$\text{trace-support extent in } d \times \prod_{r \neq d} \text{volume basis extent in } r.$$

7.3 Cell-ordering interpretation for traces

The current `TENSOR_CELL_ORDERING` is a second flattening of the same variable-local space. For traces, we should preserve this principle, but the phrase "cell ordering" becomes slightly overloaded because one factor is a skeleton-support index rather than a cell basis index.

The most conservative approach is to preserve the public enum and behavior shape while interpreting the alternate ordering as a more trace-support-centric traversal order for trace variables. In practice, issue #300 may only need to guarantee invertibility and same-size consistency for both orderings, rather than asserting a deep FE-style semantic distinction immediately.

8 Implementation Phases

We expect the implementation to proceed in the following phases.

8.1 Phase 1: trace and skeleton API scaffolding

In this phase, we will:

- extend `TensorVar` with dimension-specific trace metadata,
- add focused tests for construction and validation of dimension-specific trace variables,
- introduce explicit tensor-skeleton support in `TensorMeshTopology`, and
- add focused topology tests for tensor-skeleton enumeration and indexing.

This phase should establish the public structural vocabulary for traces before we change local layout behavior.

8.2 Phase 2: trace-local layout support

In this phase, we will:

- extend `TensorDofLayout::VariableLayout` with enough support-kind metadata to distinguish trace-support and volume-support factors,
- update `buildVariableLayout(...)` so that trace variables use skeleton-support extents in their distinguished component dimension,
- preserve deterministic variable-ordering and alternate-ordering maps, and
- add focused tests for localDofCount, ordinal maps, and invertibility for trace variables.

8.3 Phase 3: mesh/runtime integration

In this phase, we will:

- ensure that `TensorMesh` registration and validation works coherently for the new trace semantics,
- verify that `TensorGatherScatter` remains compatible with trace-local layout without unnecessary redesign, and
- add integration-style tests showing that registered trace variables produce coherent local layouts and gather/scatter sizes.

8.4 Phase 4: stabilization and polish

Finally, we will:

- review naming and comments for clarity,
- tighten validation behavior where needed,
- verify that the new support still aligns with the issue text and the broader meta-issue sequencing, and
- leave explicit room in comments or interfaces for later trace-aware operator realization.

9 Planned Test Strategy

The test strategy should remain focused and structural, since issue #300 is not yet about full side-term operator realization.

9.1 1. `TensorVar` tests

We should add tests that check:

- construction of a trace variable with an explicit trace component ordinal,
- rejection of an out-of-range trace component ordinal,
- rejection of a trace variable missing its required trace component specification,
- preservation of trace-component metadata under `copyWithNewName(...)`.

9.2 2. `TensorMeshTopology` skeleton tests

We should add tests that check:

- tensor-skeleton entity count for a chosen component dimension,
- lower-indexed-components-vary-fastest ordering of skeleton entities,
- agreement between skeleton entities and the existing slice-at-vertex interpretation,
- boundary/interior distinction at least at the component-vertex-incidence level where appropriate.

Representative examples should include at least a 2-component and ideally a 3-component tensor topology so that the "orthogonal to chosen dimension" interpretation is tested in a nontrivial setting.

9.3 3. `TensorDofLayout` trace-layout tests

We should add tests that check:

- local dof count for a trace variable equals the expected hybrid product of one vertex extent with the remaining basis extents,
- trace-local variable ordering uses lower-indexed components varying fastest,
- variable and alternate orderings remain same-size and invertible,
- per-component support-kind metadata is correctly reflected for trace variables.

9.4 4. `TensorMesh` and `TensorGatherScatter` integration tests

We should add tests that check:

- trace discretizations register successfully on `TensorMesh`,
- created dof layouts expose the expected trace-local structure,
- gather/scatter local dof counts match the new trace-local layout.

10 Files Most Likely to Change

The following existing files appear most likely to change:

- `src/include/TensorVar.h`
- `src/Mesh/TensorVar.cpp`
- `src/include/TensorMeshTopology.h`
- `src/Mesh/TensorMeshTopology.cpp`
- `src/include/TensorDofLayout.h`
- `src/Mesh/TensorDofLayout.cpp`
- `src/include/TensorMesh.h` and possibly `src/Mesh/TensorMesh.cpp`
- `src/include/TensorGatherScatter.h` and likely only lightly `src/Mesh/TensorGatherScatter.cpp`
- `unit_tests/TensorVarTests.cpp`
- `unit_tests/TensorMeshTopologyTests.cpp`
- `unit_tests/TensorDofLayoutTests.cpp`
- `unit_tests/TensorMeshTests.cpp`

- `unit_tests/TensorGatherScatterTests.cpp`
- `unit_tests/TensorVarDiscretizationTests.cpp` if constructor validation expectations change materially.

At present, we do not expect to need new files or `CMakeLists.txt` changes.

11 Risks and Design Tensions

11.1 Risk 1: overloading current component-cardinality semantics

The current `VariableLayout` assumes that every component contributes a basis cardinality. Extending it to traces introduces a second kind of factor extent. We should do this explicitly, rather than trying to smuggle vertex extents through basis-cardinality language and thereby making later operator work harder to understand.

11.2 Risk 2: ambiguity about what a trace-local coefficient represents

At the current structural stage, a trace-local coefficient block is not yet tied to a full side-basis realization path. We therefore need to be careful to describe what is implemented and what is deferred. The present work should define a coherent *ordinal and support interpretation*, not claim that all trace FE semantics are complete.

11.3 Risk 3: accidental architectural drift toward legacy side-entity models

Because legacy Camellia already has full-dimensional side and trace notions, it would be easy to import too much of that conceptual machinery. We should resist that and keep the defining model tensor-native: component skeletons tensored with remaining component topologies through explicit slice semantics.

11.4 Risk 4: future higher-dimensional generalization

The current design is intentionally shaped for orthogonal 1D components. We should choose names and interfaces that do not preclude future higher-dimensional spatial components, but we should not overgeneralize prematurely. For issue #300, the 1D orthogonal setting is sufficient.

12 Bounded Unknowns

A few bounded unknowns remain, but they do not appear to block implementation.

- We may need to decide whether the trace support in a component dimension should use all active vertices or only those that are incident to at least one active cell in the intended slice sense. In practice, these should coincide for the current topology representation, but it is worth verifying during implementation.
- We may need to decide how much explicit boundary/interior metadata belongs in the first tensor-skeleton helper type. My expectation is that a modest first pass is sufficient.
- We may discover that one or two helper queries should be added to `ComponentMeshTopology1D` for readability, even though the core incidence data is already present.

These appear to be implementation details rather than design blockers.

13 Stop Conditions During Implementation

We should stop and ask for clarification if either of the following occurs:

1. the current issue scope turns out to require a major redesign of the existing tensor-native local coefficient-space interpretation rather than an extension of it, or
2. later trace-aware operator realization requirements force a materially different trace-support model than the present slice-based "component-d skeleton times retained active tuples" construction during the present implementation.

   At present, neither appears likely.

14 Expected Outcome

If the above plan succeeds, the result of issue #300 should be that the tensor-native stack gains an explicit and dimension-specific trace-support substrate. In particular, we expect to have:

- trace variables that know which tensor component dimension they belong to,
- explicit tensor-native skeleton support objects or views for traces orthogonal to a chosen component dimension,
- an initial trace-local layout interpretation compatible with the current tensor-native local coefficient-space story, and
- focused tests demonstrating that the geometric and ordinal interpretation matches the intended component-skeleton-with-slice-retained-tuples model.

This should close the next major architectural gap after the volume-only FE-aware apply path from issue #299, while leaving a clean path toward later work on trace-aware basis selection, side quadrature, mixed volume/trace terms, and eventually trace-aware matrix-free or PAMatrix-oriented realization.